

MWORKS.Syslab 2023b

并行计算工具箱

陈久宁

苏州同元软控信息技术有限公司

2025年5月23日



TONGYUAN
Software and Control

@苏州同元软控信息技术有限公司版权所有，
版权所有，侵权必究

传播或以其他方式使用

目录

1. 并行计算导引

2. CPU 指令集并行

3. 异步编程基础

4. 任务间通信

01

并行计算导引

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用

版权所有，侵权必究

1.1 并行计算导引：概述

并行计算是一门通过更好地调度各种计算资源来实现计算加速的手段。

常见的基本并行策略有：

并行策略	说明	加速比	调度开销	优点	缺点
CPU 指令级并行	调用硬件级别的指令级并行指令	1-8 倍	1-5ns	开销极低，可以获得近乎免费的加速，可以在底层代码中直接使用	依赖硬件支持，不可移植；对数据布局及运算有要求
异步流水线	通过将完整的任务拆分为逻辑独立的小任务，以流水线的方式执行	0.9-5 倍	0.5-1μs	降低不同计算设备的等待时间，提高计算资源的利用率	编程模式与常见的顺序编程的方式并不相同，需要学习与训练
单机多线程	通过多线程的方式调度多个CPU核心	0.9-N 倍	2-10μs	可以调度全部的 CPU 资源	存在不算大的开销，在数据竞争与共享问题上需要特殊处理
单机多进程	通过多进程的方式调度多个CPU核心	0.8-N 倍	50-300μs	可以调度全部的 CPU 资源；数据隔离	进程创建存在较大开销，进程间通信也存在一定开销
GPU异构计算	对于特定任务调度专用的计算设备来获得比 CPU 更高的性能	10-10000 倍	5-20μs	GPU是为并行计算设计的设备，在大规模并行中可以以更低成本获得远高于CPU的加速比	需要特殊的设备；需要特殊的编程模式和专用的编程工具；与CPU 主内存并不共享，数据传输开销需要特别处理
分布式计算	调度更多的计算机	0.5-N 倍	0.1-2s	可以调度更多的计算设备	分布式计算的编程范式复杂；需要维护集群环境；需要考虑稳定性与停机问题

如何根据实际任务场景选择不同的并行策略，以及不同并行策略之间的组合，是利用好并行计算的核心。



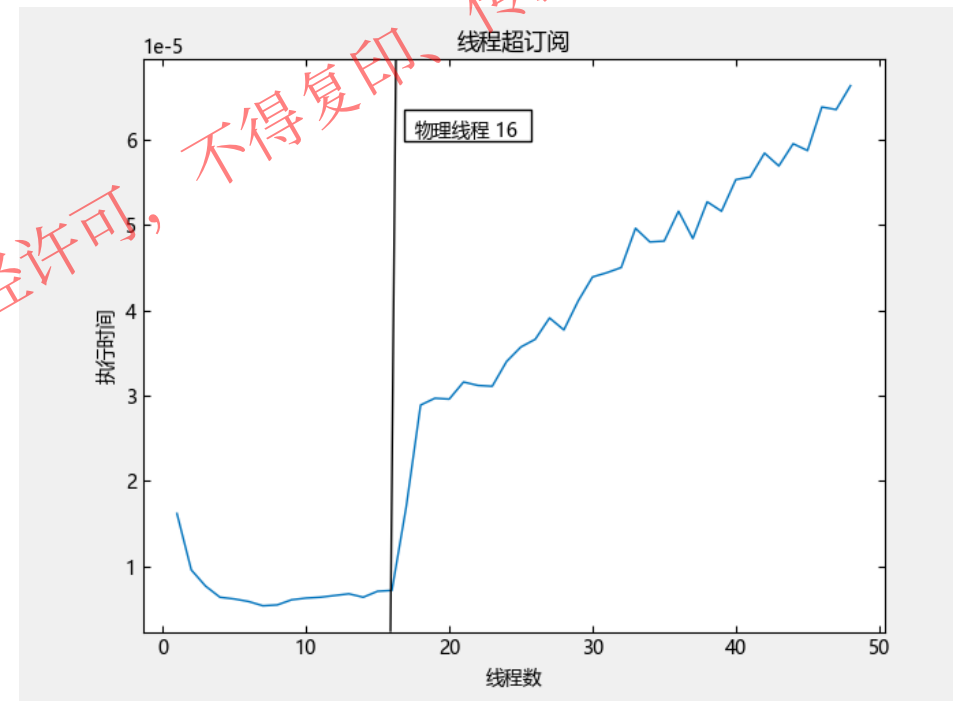
1.2 并行计算导引：关于并行计算的误解

(不恰当的) 并行可能会让代码变得更慢

例：线程过度订阅 (oversubscription) 导致性能下降

```
function f(n)
    S = zeros(n)
    Threads.@threads for i in 1:n
        S[i] = sum(rand(256))
    end
    return sum(S)
end
```

多线程独立运算



物理资源是有限的：线程订阅过多时，会出现不同线程争抢同一物理资源的现象。

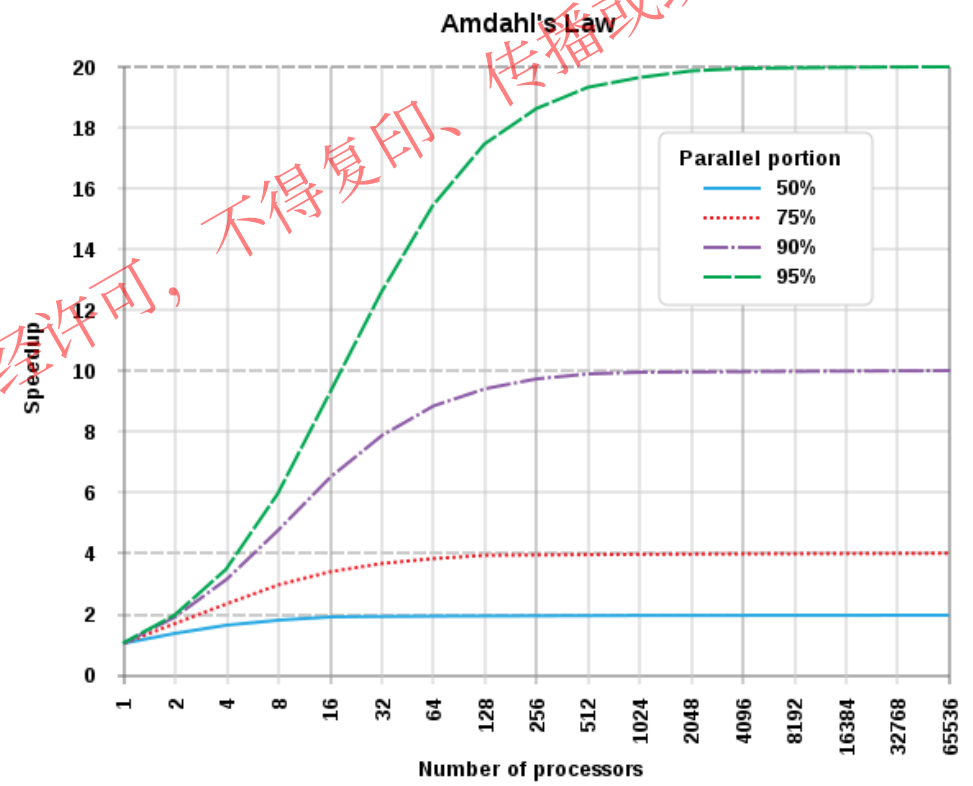
Tip: 通过 `julia -t<n>` 来以多线程方式启动 Julia，如: `julia -t4`

1.2 并行计算导引：关于并行计算的误解

并行计算的性能提升不是线性的：阿姆达尔定律

$$\text{加速比} = \frac{1}{(1-p)+P/N}$$

p: 并行效率 (parallel portion)
N: 并行规模



并行是有开销的，如：进程间通讯、数据复制、状态同步、网络、任务拆分。

1.2 并行计算导引：关于并行计算的误解

(不恰当的) 并行不仅可能会慢，还可能会错

例：数据竞争 (data-racing) 可能会导致结果错误

```
function simple_sum(A)
    rst = zero(eltype(A))
    for x in A
        rst += x
    end
    return rst
end
```

串行求和

```
function parallel_sum(A)
    rst = zero(eltype(A))
    Threads.@threads for x in A
        rst += x
    end
    return rst
end
```

多线程求和

PS C:\Users\...\Documents\TongYuan\并行计算> julia -t4 .\simple_sum.jl

[Info: 内置求和: 5000000050000000 时间: 1.661661661661662e-8

[Info: 串行求和: 5000000050000000 时间: 1.7717717717717717e-8

[Info: 多线程(N=4) 求和: 937538162791466 时间: 2.4271388

02 CPU 指令集并行

©苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用

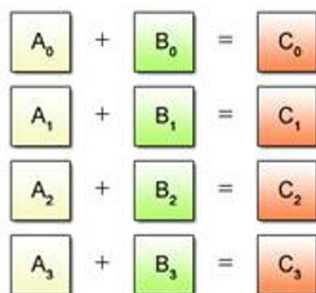
@苏州同元软控信息技术有限公司
版权所有，侵权必究

2.1 CPU 指令集并行: SIMD 是什么

SIMD(Single Instruction, Multiple Data) 即单指令多数据, 是一种在 CPU 内并行计算的方法。通过这种方法, 可以同时多个数据元素执行单个操作。

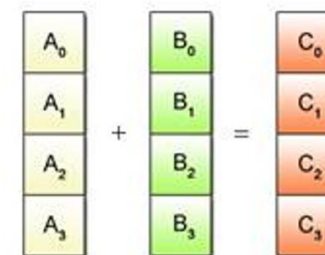
4次标量运算

(a) Scalar Operation



1次向量运算

(b) SIMD Operation



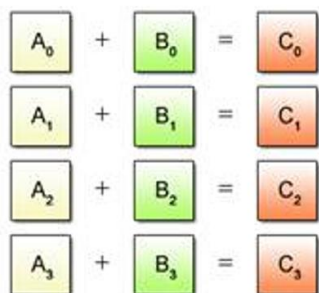
Tip: 人们有时会用向量化(vectorization)来表达对 SIMD 等向量化指令的使用

2.1 CPU 指令集并行: SIMD是什么

SIMD(Single Instruction, Multiple Data) 即单指令多数据, 是一种在 CPU 内并行计算的方法。通过这种方法, 可以同时多个数据元素执行单个操作。

4次加法运算

(a) Scalar Operation



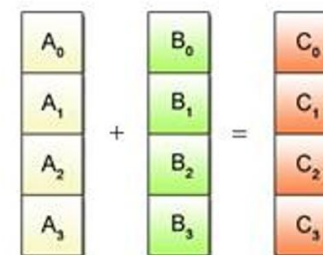
```
function simple_sum(A)
    rst = zero(eltype(A))
    for x in A
        rst += x
    end
    return rst
end
```

```
julia> using BenchmarkTools
julia> A = rand(64, 64);
julia> @btime simple_sum(A);
1.560 μs (1 allocation: 16 bytes)
julia> @btime simd_sum(A);
406.000 ns (1 allocation: 16 bytes)
```

SIMD 带来了约4倍性能提升

1次加法运算

(b) SIMD Operation



```
function simd_sum(A)
    rst = zero(eltype(A))
    @simd for x in A
        rst += x
    end
    return rst
end
```

@simd 宏用于告诉 Julia: 请尽量将这段代码编译为 SIMD 的版本

Tip: 人们有时会用向量化(vectorization)来表达对 SIMD 等向量化指令的使用

2.2 CPU 指令集并行: SIMD 加速比估计

SIMD 的加速比取决于数据类型的位宽和 CPU 的架构

大部分 CPU 支持 256 位 SIMD, 即一次向量化运算的总位宽

4 x Float64 = 256 位宽

```
julia> A = rand(Float64, 64, 64);  
  
julia> @btime simple_sum(A);  
1.590 μs (1 allocation: 16 bytes)  
  
julia> @btime simd_sum(A);  
414.070 ns (1 allocation: 16 bytes)
```

加速比: 4x

8 x Float32 = 256 位宽

```
julia> A = rand(Float32, 64, 64);  
  
julia> @btime simple_sum(A);  
1.590 μs (1 allocation: 16 bytes)  
  
julia> @btime simd_sum(A);  
210.714 ns (1 allocation: 16 bytes)
```

加速比: 8x

AVX-512 是 Intel 提出的 512 位版本的对 SIMD 指令的扩展, 但是下游的接受程度不高

Tip: 几乎所有 CPU 均不支持 16 位浮点运算

2.3 CPU 指令集并行: SIMD 的使用限制

即使加了 `@simd` 宏, Julia 也不一定会编译出带 SIMD 的版本

SIMD 代码中不允许条件分支, 即: `if` 判断或 `?:` 三元运算符均不允许出现

```
function simple_dot(A, B)
    @assert axes(A) == axes(B)
    rst = zero(eltype(A))
    for i in eachindex(A)
        rst += A[i] * B[i]
    end
    return rst
end
```

向量内积

```
function simd_dot(A, B)
    @assert axes(A) == axes(B)
    rst = zero(eltype(A))
    @simd for i in eachindex(A)
        rst += A[i] * B[i]
    end
    return rst
end
```

向量内积: 带 `@simd` 宏

没有性能差异
(SIMD 未生效)

```
julia> A, B = rand(4096), rand(4096);

julia> @btime simple_dot(A, B);
1.650 μs (1 allocation: 16 bytes)

julia> @btime simd_dot(A, B);
1.670 μs (1 allocation: 16 bytes)
```

Tip: NumPy dot 运算的时间为 11.6 μs

2.3 CPU 指令集并行: SIMD 的使用限制

即使加了 `@simd` 宏, Julia 也不一定会编译出带 SIMD 的版本

SIMD 代码中不允许条件分支, 即: `if` 判断或 `?:` 三元运算符均不允许出现

```
function simple_dot(A, B)
    @assert axes(A) == axes(B)
    rst = zero(eltype(A))
    for i in eachindex(A)
        rst += A[i] * B[i]
    end
    return rst
end
```

向量内积

```
function simd_dot(A, B)
    @assert axes(A) == axes(B)
    rst = zero(eltype(A))
    @simd for i in eachindex(A)
        rst += A[i] * B[i]
    end
    return rst
end
```

向量内积: 带 `@simd` 宏

```
function simd_inbounds_dot(A, B)
    @assert axes(A) == axes(B)
    rst = zero(eltype(A))
    @inbounds @simd for i in eachindex(A)
        rst += A[i] * B[i]
    end
    return rst
end
```

向量内积: 带 `@simd` 与 `@inbounds` 宏

~3x 性能差异
(SIMD 生效)

```
julia> A, B = rand(4096), rand(4096);
julia> @btime simple_dot(A, B);
1.650 μs (1 allocation: 16 bytes)
julia> @btime simd_inbounds_dot(A, B);
555.191 ns (1 allocation: 16 bytes)
```

Julia 矩阵取值时默认会进行下标越界检查, 该检查本质上是一个 `if` 判断, 阻碍了 SIMD 生效。 `@inbounds` 宏可以用于取消该检查从而让 SIMD 生效

警告: 不要滥用 `@inbounds`: 只在确定不会出现下标越界的情况下才使用它

Tip: Julia 内置 dot 运算的时间为 481.538 ns (1 allocation: 16 bytes)

2.3 CPU 指令集并行: SIMD 的使用限制

即使加了 `@simd` 宏, Julia 也不一定会编译出带 SIMD 的版本

SIMD 代码中不允许条件分支, 即: `if` 判断或 `?:` 三元运算符均不允许出现

```
function simple_sum_if(A, B)
    @assert axes(A) == axes(B)
    rst = zero(eltype(A))
    @inbounds for i in eachindex(A)
        a, b = A[i], B[i]
        if a * b > 0.5
            rst += a + b
        end
    end
    return rst
end
```

条件求和

```
function simd_sum_if(A, B)
    @assert axes(A) == axes(B)
    rst = zero(eltype(A))
    @inbounds @simd for i in eachindex(A)
        a, b = A[i], B[i]
        if a * b > 0.5
            rst += a + b
        end
    end
    return rst
end
```

条件求和: 带 `@simd` 宏

没有性能差异
(SIMD 未生效)

```
julia> A, B = rand(4096), rand(4096);

julia> @btime simple_sum_if(A, B);
4.286 μs (1 allocation: 16 bytes)

julia> @btime simd_sum_if(A, B);
4.371 μs (1 allocation: 16 bytes)
```

Tip: 有时编译器也会识别普通的 `if-else` 分支, 并为其编译 SIMD 版本

2.3 CPU 指令集并行: SIMD 的使用限制

即使加了 `@simd` 宏, Julia 也不一定会编译出带 SIMD 的版本

SIMD 代码中不允许条件分支, 即: `if` 判断或 `?:` 三元运算符均不允许出现

```
function simple_sum_if(A, B)
    @assert axes(A) == axes(B)
    rst = zero(eltype(A))
    @inbounds for i in eachindex(A)
        a, b = A[i], B[i]
        if a * b > 0.5
            rst += a + b
        end
    end
    return rst
end
```

条件求和

```
function simd_sum_if(A, B)
    @assert axes(A) == axes(B)
    rst = zero(eltype(A))
    @inbounds @simd for i in eachindex(A)
        a, b = A[i], B[i]
        if a * b > 0.5
            rst += a + b
        end
    end
    return rst
end
```

条件求和: 带 `@simd` 宏

```
function simd_sum_ifelse(A, B)
    @assert axes(A) == axes(B)
    rst = zero(eltype(A))
    @inbounds @simd for i in eachindex(A)
        a, b = A[i], B[i]
        rst += ifelse(a * b > 0.5, a + b, zero(rst))
    end
    return rst
end
```

`ifelse` 是一个无分支形式的条件判断函数。它与正常的 `if-else` 分支的差别在于: `ifelse` 两种情况下的值都会计算, 而 `if-else` 会根据条件选择性地计算一种情况下地值。

优化思路: 引入额外计算来消除分支, 从而换取 SIMD 的开启

注意: `ifelse` 几乎只用于底层代码; 高层代码上没有收益

~5.5x 性能差异
(SIMD 生效)

```
julia> A, B = rand(4096), rand(4096);

julia> @btime simple_sum_if(A, B);
4.286 μs (1 allocation: 16 bytes)

julia> @btime simd_sum_ifelse(A, B);
742.857 ns (1 allocation: 16 bytes)
```

Tip: 常见运算的 CPU 指令时间 <http://ithare.com/infographics-operation-costs-in-cpu-clock-cycles/>

2.3 CPU 指令集并行: SIMD 的使用限制

即使加了 `@simd` 宏, Julia 也不一定会编译出带 SIMD 的版本

只有非常有限的基本指令支持 SIMD, 如: 四则运算、位运算

```
function simple_sum_mod(A)
    rst = zero(elttype(A))
    for x in A
        rst += x % 3
    end
    return rst
end
```

带模求和

```
function simd_sum_mod(A)
    rst = zero(elttype(A))
    @simd for x in A
        rst += x % 3
    end
    return rst
end
```

带模求和: 带 `@simd` 宏

没有性能差异
(SIMD 未生效)

```
julia> A = rand(1:255, 64, 64);

julia> @btime simple_sum_mod(A);
1.660 μs (1 allocation: 16 bytes)

julia> @btime simd_sum_mod(A);
1.660 μs (1 allocation: 16 bytes)
```

2.3 CPU 指令集并行: SIMD 的使用限制

即使加了 `@simd` 宏, Julia 也不一定会编译出带 SIMD 的版本

只有非常有限的基本指令支持 SIMD, 如: 四则运算、位运算

```
function simple_sum_mod(A)
    rst = zero(eltype(A))
    for x in A
        rst += x % 3
    end
    return rst
end
```

带模求和

```
function simd_sum_mod(A)
    rst = zero(eltype(A))
    @simd for x in A
        rst += x % 3
    end
    return rst
end
```

带模求和: 带 `@simd` 宏

```
function mod3_lookup(n)
    lookup = Vector{Int}(undef, n)
    for i in 1:n
        lookup[i] = i % 3
    end
    return lookup
end
const LOOKUP_MOD3 = mod3_lookup(255)

function simd_sum_mod_lookup(A)
    rst = zero(eltype(A))
    @simd for x in A
        rst += @inbounds LOOKUP_MOD3[x]
    end
    return rst
end
```

基于查找表的带模求和

优化思路: 通过引入一些合理的限制与假设, 不支持 SIMD 的运算可以转换为 SIMD 友好的运算

```
julia> A = rand(1:255, 64, 64);

julia> @btime simple_sum_mod(A);
1.660 μs (1 allocation: 16 bytes)

julia> @btime simd_sum_mod_lookup(A);
844.928 ns (1 allocation: 16 bytes)
```

~2x性能差异
(SIMD 生效)

2.4 CPU 指令集并行：自动 SIMD 推断

即使没有 `@simd` 宏，Julia 也可能会自动推断并编译 SIMD 版本

```
function simple_sum(A)
    rst = zero(eltype(A))
    for x in A
        rst += x
    end
    return rst
end
```

简单求和

```
function simd_sum(A)
    rst = zero(eltype(A))
    @simd for x in A
        rst += x
    end
    return rst
end
```

简单求和：带 `@simd` 宏

没有性能差异
(SIMD 生效)

```
julia> A = rand(1:255, 64, 64);

julia> @btime simple_sum(A);
268.439 ns (1 allocation: 16 bytes)

julia> @btime simd_sum(A);
269.206 ns (1 allocation: 16 bytes)
```

Tip: Julia 编译器很多时候会自动推断并补上 `@inbounds` 和 `@simd`

2.4 CPU 指令集并行：自动 SIMD 推断

即使没有 `@simd` 宏，Julia 也可能会自动推断并编译 SIMD 版本

```
function simple_sum(A)
    rst = zero(eltype(A))
    for x in A
        rst += x
    end
    return rst
end
```

简单求和

Julia 会在不改变数值结果的前提下尝试自动补充和生成 SIMD 版本代码

```
function simd_sum(A)
    rst = zero(eltype(A))
    @simd for x in A
        rst += x
    end
    return rst
end
```

简单求和：带 `@simd` 宏

没有性能差异
(SIMD 生效)

```
julia> A = rand(1:255, 64, 64);

julia> @btime simple_sum(A);
268.439 ns (1 allocation: 16 bytes)

julia> @btime simd_sum(A);
269.206 ns (1 allocation: 16 bytes)
```

Int 类型：加法满足交换律

```
julia> A = rand(Float64, 64, 64);

julia> @btime simple_sum(A);
1.600 μs (1 allocation: 16 bytes)

julia> @btime simd_sum(A);
415.578 ns (1 allocation: 16 bytes)
```

Float64 类型：加法不满足交换律

Tip: Julia 编译器很多时候会自动推断并补上 `@inbounds` 和 `@simd`

通过 `@code_llvm` 来检查 SIMD 是否真正开启

```
julia> A = rand(1:256, 64, 64);

julia> @code_llvm simple_sum(A)
; @ c:\Users\...\Documents\TongYuan\并行计算\simple_sum.jl:3 within `simple_sum`
; Function Attrs: uwtable
define i64 @julia_simple_sum_5262({}* nonnull align 8 @array, i8* ptr) {
top:
; @ c:\Users\jc\Documents\TongYuan\并行计算\simple_sum.jl:835 within `iterate` @ array.jl:835
; [ @ array.jl:215 within `length`
    %1 = bitcast {}* %0 to { i8*, i64, i16, i16, i16, i16, i16, i16 }
    ...
```

当你在 LLVM 代码中看到了类似于 `<2 x i64> <4 x float> vector.body` 之类的标记时，说明对应的代码 SIMD 启用了

```
vector.ph: ; preds = %L37.preheader
  %n.vec = and i64 %7, -4
  %ind.end = or i64 %n.vec, 1
  %ind.end21 = or i64 %n.vec, 2
  br = insertelement <2 x i64> <i64 poison, i64 0>, i64 %6, i32 0
  %8 = label %vector.body
```

```
vector.body:
; preds = %vector.body, %vector.ph
%index = phi i64 [ 0, %vector.ph ], [ %index.next, %vector.body ]
%vec.ph = phi <2 x i64> [%8, %vector.ph ], [ %13, %vector.body ]
%vec.ph122 = phi <2 x i64> [ %zeroinitializer, %vector.ph ], [ %14, %vector.body ]
%offset.idx = or i64 %index, 1
; | @ array.jl:861 within `getindex`
%9 = getelementptr inbounds i64, i64* %5, i64 %offset.idx
%10 = bitcast i64* %9 to <2 x i64>*
%wide.load = load <2 x i64>, <2 x i64>* %10, align 8
%11 = getelementptr inbounds i64, i64* %9, i64 2
%12 = bitcast i64* %11 to <2 x i64>*
%wide.load24 = load <2 x i64>, <2 x i64>* %12, align 8
; LL
; | @ int.jl:87 within `+`
%13 = add <2 x i64> %vec.ph, %wide.load
%14 = add <2 x i64> %vec.ph122, %wide.load24
%index.next = add i64 %index, 4
%15 = icmp eq i64 %index.next, %n.vec
br i1 %15, label %middle_block, label %vector.body
```

SIMD 确实生效了

```
julia> A = rand{Float64, 64, 64};  
  
julia> @code_llvm simple_sum(A)  
... @ c:\Users\TongYuan\Documents\并行计算\simple_sum.jl:3 within `simple_sum`  
table  
_simple_sum_5270({}* nonnull align 16 dereferenceable(40) %0) #0 {  
... documents\TongYuan\并行计算\simple_sum.jl:5 within `simple_sum`
```

```

L37:                                ; preds = %L37, %L18
    %8 = phi i64 [ %value_phi418, %L37 ], [ 1, %L18 ]
    %9 = phi double [ %13, %L37 ], [ %7, %L18 ]
    %value_phi418 = phi i64 [ %12, %L37 ], [ 2, %L18 ]
; | @ array.jl:861 within `getindex`
    %10 = getelementptr inbounds double, double* %5, i64 %8
    %11 = load double, double* %10, align 8
; |
; | @ int.jl:87 within `+`
    %12 = add nuw nsw i64 %value_phi418, 1
; LL
; | @ float.jl:399 within `+`
    %13 = fadd double %11, %9
; |
; | @ array.jl:835 within `iterate`
; | @ int.jl:483 within `<` @ int.jl:476
    %exitcond.not = icmp eq i64 %value_phi418, %3
; |
    br i1 %exitcond.not, label %L43, label %L37

```

SIMD 没生效

Tip: 大多数时候并不需要真的看懂 LLVM 代码就可以诊断 SIMD 是否生效

2.6 CPU 指令集并行：小结

SIMD 是 CPU 指令集并行：它可以带来近乎免费的性能加速，但使用条件比较严格

最佳实践：

- 一般 SIMD 仅发生在底层基础代码：高层代码几乎不可能满足 SIMD 的要求
 - `@simd` 宏只在确实有效的时候才添加
 - `@inbounds` 宏只在确定下标不会越界，并且能够带来可观性能优势的时候才添加
 - `@simd` 只可能出现在嵌套 `for` 循环的最内层循环
- 通过性能对比 (`@btime`) 来验证 `@simd` 的加速效果
- 通过 `@code_llvm` 来检查生成的代码中 SIMD 是否生效
- 升级 Julia 版本：编译器会做非常多好的 SIMD 优化

更多阅读：

- `SIMD.jl` 允许手动的 SIMD 代码编写：对于特定底层算法实现可以带来更激进的优化策略
- `LoopVectorization.jl` 提供 `@turbo` 宏：包含 AVX 以及 `for` 循环优化等综合性手段

2.6 CPU 指令集并行：小结

SIMD 是 CPU 指令集并行：它可以带来近乎免费的性能加速，但使用条件比较严格

最佳实践：

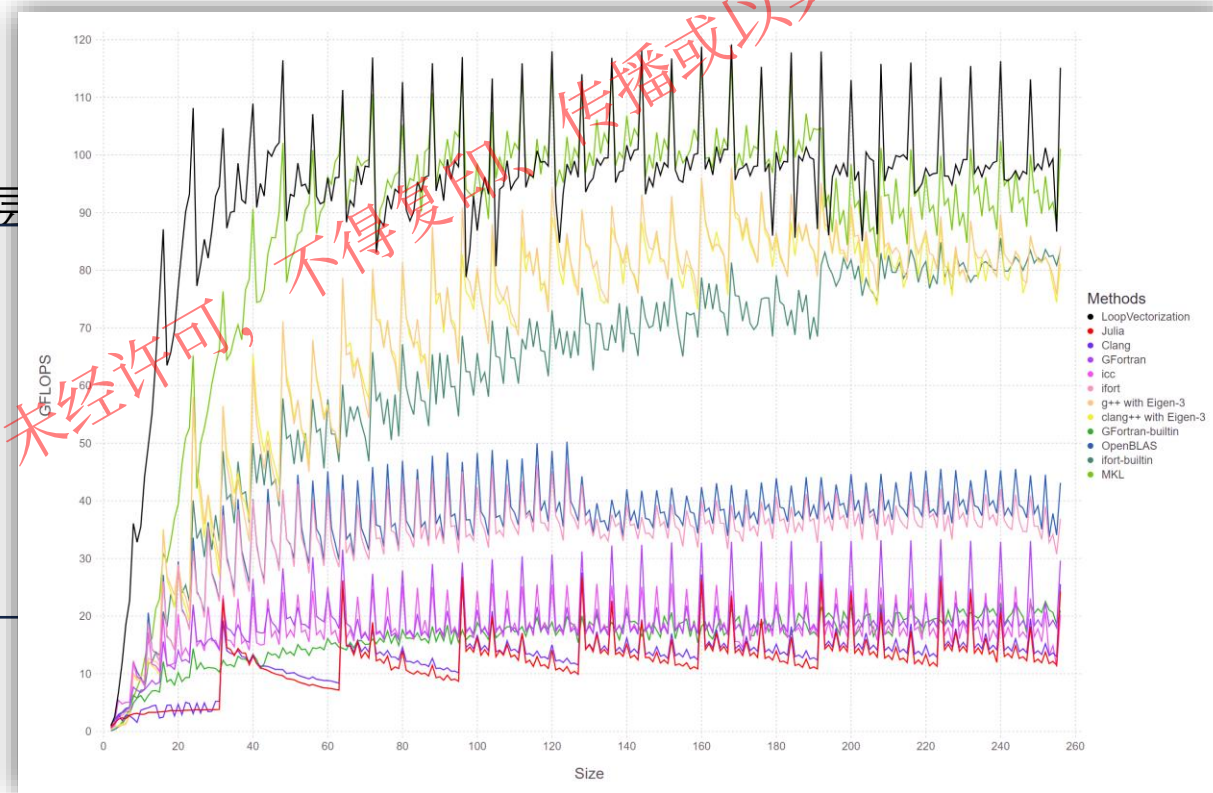
- 一般 SIMD 仅发生在底层基础代码：高层

```
function A_mul_B!(C, A, B)
    @turbo for n ∈ indices((C,B), 2), m ∈ indices((C,A), 1)
        Cmn = zero(eltype(C))
        for k ∈ indices((A,B), (2,1))
            Cmn += A[m,k] * B[k,n]
        end
        C[m,n] = Cmn
    end
end
```

LoopVectorization.jl

更多阅读：

- SIMD.jl 允许手动的 SIMD 代码编写：对于特定底层算法实现可以带来更激进的优化策略
- LoopVectorization.jl 提供 @turbo 宏：包含 AVX 以及 for 循环优化等综合性手段



Tip: 基于 LoopVectorization 的矩阵乘法 https://juliasimd.github.io/LoopVectorization.jl/stable/examples/matrix_multiplication/

2.6 CPU 指令集并行：小结

SIMD 是 CPU 指令集并行：它可以带来近乎免费的性能加速，但使用条件比较严格

最佳实践：

- 一般 SIMD 仅发生在底层基础代码：高层代码几乎不可能满足 SIMD 的要求
 - `@simd` 宏只在确实有效的时候才添加
 - `@inbounds` 宏只在确定下标不会越界 并且能够带来可观性能优势的时候才添加
 - `@simd` 只可在 Julia 中：人人都可以尝试写底层代码
- 通过性能对比 (`@btime`) 来验证 `@simd` 的加速效果
- 通过 `@code_llvm` 来检查生成的代码中 SIMD 是否生效
- 升级 Julia 版本：编译器会做非常多好的 SIMD 优化

更多阅读：

- `SIMD.jl` 允许手动的 SIMD 代码编写：对于特定底层算法实现可以带来更激进的优化策略
- `LoopVectorization.jl` 提供 `@turbo` 宏：包含 AVX 以及 for 循环优化等综合性手段

03 异步编程基础

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用

版权所有，侵权必究

3.1 异步编程基础：背景

在实际编程中，一个程序中的不同子任务会调度不同的硬件单元。例如：
调度 CPU 资源进行数值计算；调度磁盘资源进行 IO 计算；调度网卡进行通讯；调度显卡进行计算。

```
# 定时函数：占用 CPU 资源
function busywait(seconds=2)
    tstart = time_ns()
    while (time_ns() - tstart) / 1e9 < seconds
        continue
    end
    return (time_ns() - tstart) / 1e9
end
```

```
# 定时函数：不占用 CPU 资源
sleep(2)
```

该函数由5个 CPU 密集型任务以及5个 IO 密集型任务组成

```
function f_sync()
    for i in 1:5
        busywait(2)
        sleep(1)
    end
    return nothing
end
```

```
julia> @time f_sync()
15.035560 seconds (2.47 k allocations: 154.938 KiB)
```

3.1 异步编程基础：背景

在实际编程中，一个程序中的不同子任务会调度不同的硬件单元。例如：
调度 CPU 资源进行数值计算；调度磁盘资源进行 IO 计算；调度网卡进行通讯；调度显卡进行计算。

该函数由5个 CPU 密集型任务以及5个 IO 密集型任务组成

```
# 定时函数：占用 CPU 资源
function busywait(seconds=2)
    tstart = time_ns()
    while (time_ns() - tstart) / 1e9 < seconds
        continue
    end
    return (time_ns() - tstart) / 1e9
end
```

```
# 定时函数：不占用 CPU 资源
sleep(2)
```

```
function f_sync()
    for i in 1:5
        busywait(2)
        sleep(1)
    end
end
```



15.035560 seconds (2.47 k allocations: 154.938 KiB)

3.1 异步编程基础：背景

在实际编程中，一个程序中的不同子任务会调度不同的硬件单元。例如：
调度 CPU 资源进行数值计算；调度磁盘资源进行 IO 计算；调度网卡进行通讯；调度显卡进行计算。

该函数由5个 CPU 密集型任务以及5个 IO 密集型任务组成

```
# 定时函数：占用 CPU 资源
function busywait(seconds=2)
    tstart = time_ns()
    while (time_ns() - tstart) / 1e9 < seconds
        continue
    end
    return (time_ns() - tstart) / 1e9
end
```

```
# 定时函数：不占用 CPU 资源
sleep(2)
```

```
function f_sync()
    for i in 1:5
        busywait(2)
        sleep(1)
    end
end
```

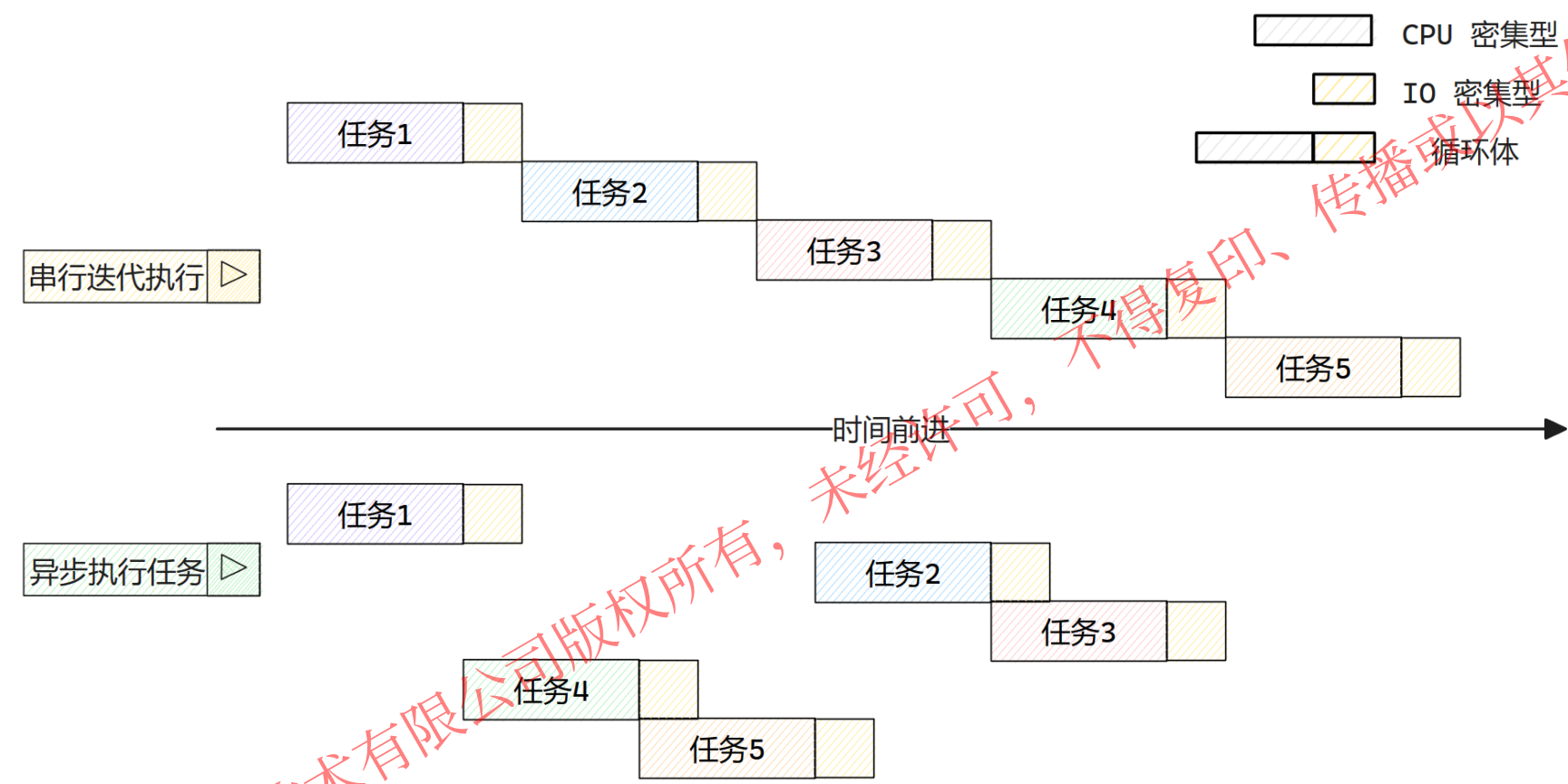


()
15.035560 seconds (2.47 k allocations: 154.938 KiB)

如果两个任务使用的是不同的硬件资源，那么它们完全可以同时执行



3.2 异步编程基础：背景



3.2 异步编程基础：异步任务

任务 (Task) 的创建与调度

Task 用于创建任务：它接受一个无参函数作为对象

```
julia> t = Task(busywait)
Task (runnable) @0x0000000094dbd750
```

无参函数即不需要任何输入就可以直接执行的函数。
busywait 是一个无参函数，因为 busywait() 可以直接执行

对于有参函数，可以通过 @task 宏来辅助任务的创建

```
julia> t = @task busywait(2)
Task (runnable) @0x0000000094de1180
```

等价于 `t = Task(()->busywait(2))`

通过 schedule 函数来调度任务的执行

```
julia> schedule(t)
Task (done) @0x0000000094dbc5e0
```

schedule 只能调度状态为 runnable 的任务

@time 测试的是任务调度的时间（而非任务执行的时间）

```
julia> @time schedule(Task(busywait))
0.000006 seconds (4 allocations: 576 bytes)
Task (done) @0x0000000094de07d0
```

schedule 仅仅只是将任务加到调度器，它并不必然保证任务已经开始执行

wait 等待任务结束

```
julia> @time wait(schedule(@task busywait(2)))
2.006947 seconds (15.99 k allocations:
936.961 KiB, 0.34% compilation time)
```

schedule 仅仅只是将任务加到调度器，它并不必然保证任务已经开始执行

istaskstarted, istaskdone, istaskfailed 等函数用于任务状态检查。yield 用于让出当前任务所使用的资源

3.2 异步编程基础：异步任务

异步任务 (Task) 的创建与调度

@async 用于创建一个任务并在后台执行它

```
julia> @time @async busywait(2)
0.000020 seconds (26 allocations: 1.969 KiB)
Task (done) @0x0000000092b7b650
```

@async busywait(2) 等价于 schedule(@task busywait(2))

@sync 用于等待一个 @async 任务的结束

```
julia> @time @sync @async busywait(2)
2.003988 seconds (15.93 k allocations:
933.539 KiB, 0.19% compilation time)
Task (done) @0x00000000b9269b30
```

@sync 会自动 wait 所有由 @async 创建的任务

3.2 异步编程基础：异步任务

异步任务 (Task) 的创建与调度

@async 用于创建一个任务并在后台执行它

```
julia> @time @async busywait(2)
0.000020 seconds (26 allocations: 1.969 KiB)
Task (done) @0x0000000092b7b650
```

@async busywait(2) 等价于 schedule(@task busywait(2))

@sync 用于等待一个 @async 任务的结束

```
julia> @time @sync @async busywait(2)
2.003988 seconds (15.93 k allocations:
933.539 KiB, 0.19% compilation time)
Task (done) @0x00000000b9269b30
```

@sync 会自动 wait 由 @async 创建的任务

```
function f()
    @sync begin
        @async sleep(1)
        @async busywait(2)
        @async sleep(1)
    end
    return nothing
end
```

```
julia> @time f()
3.004885 seconds (90 allocations: 3.906 KiB)
```

3.2 异步编程基础：异步任务

异步任务 (Task) 的创建与调度

@async 用于创建一个任务并在后台执行它

```
julia> @time @async busywait(2)
0.000020 seconds (26 allocations: 1.969 KiB)
Task (done) @0x0000000092b7b650
```

@async busywait(2) 等价于 schedule(@task busywait(2))

@sync 用于等待一个 @async 任务的结束

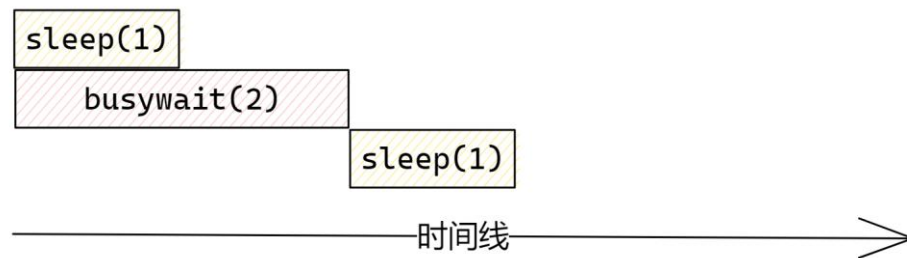
```
julia> @time @sync @async busywait(2)
2.003988 seconds (15.93 k allocations:
933.539 KiB, 0.19% compilation time)
Task (done) @0x00000000b9269b30
```

@sync 会自动 wait 由 @async 创建的任务

并发 (concurrency): 多个任务都处于被调度执行的状态

```
function f()
    @sync begin
        @async sleep(1)
        @async busywait(2)
        @async sleep(1)
    end
    return nothing
end
```

```
julia> @time f()
3.004885 seconds (90 allocations: 3.906 KiB)
```



sleep 函数并不占用 CPU 资源，因此可以并发执行

并发并不意味着并行 (concurrency is not parallelism)。并发意味着程序有并行的可能性，但很可能同时只有一个任务在执行。并行指的是同一时刻存在多个任务同时执行。

3.2 异步编程基础：异步任务

以下几种调度策略的运行时间分别是多少？为什么？

```
function f()  
  @sync begin  
    @async begin  
      busywait(2)  
      sleep(1)  
      sleep(1)  
    end  
  end  
  return nothing  
end
```

```
function f()  
  @sync begin  
    @async busywait(2)  
    @async sleep(1)  
    @async sleep(1)  
  end  
  return nothing  
end
```

```
function f()  
  @sync begin  
    @async sleep(1)  
    @async sleep(4)  
    @async busywait(3)  
  end  
  return nothing  
end
```

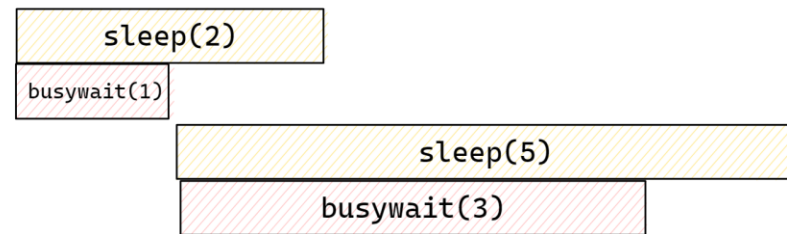
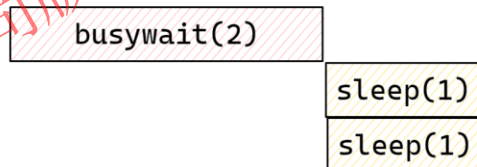
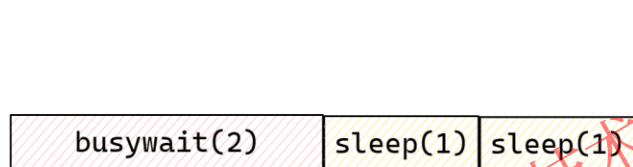
3.2 异步编程基础：异步任务

以下几种调度策略的运行时间分别是多少？为什么？

```
function f()
  @sync begin
    @async begin
      busywait(2)
      sleep(1)
      sleep(1)
    end
  end
  return nothing
end
```

```
function f()
  @sync begin
    @async busywait(2)
    @async sleep(1)
    @async sleep(1)
  end
  return nothing
end
```

```
function f()
  @sync begin
    @async sleep(2)
    @async busywait(1)
    @async sleep(5)
    @async busywait(3)
  end
  return nothing
end
```



时间线

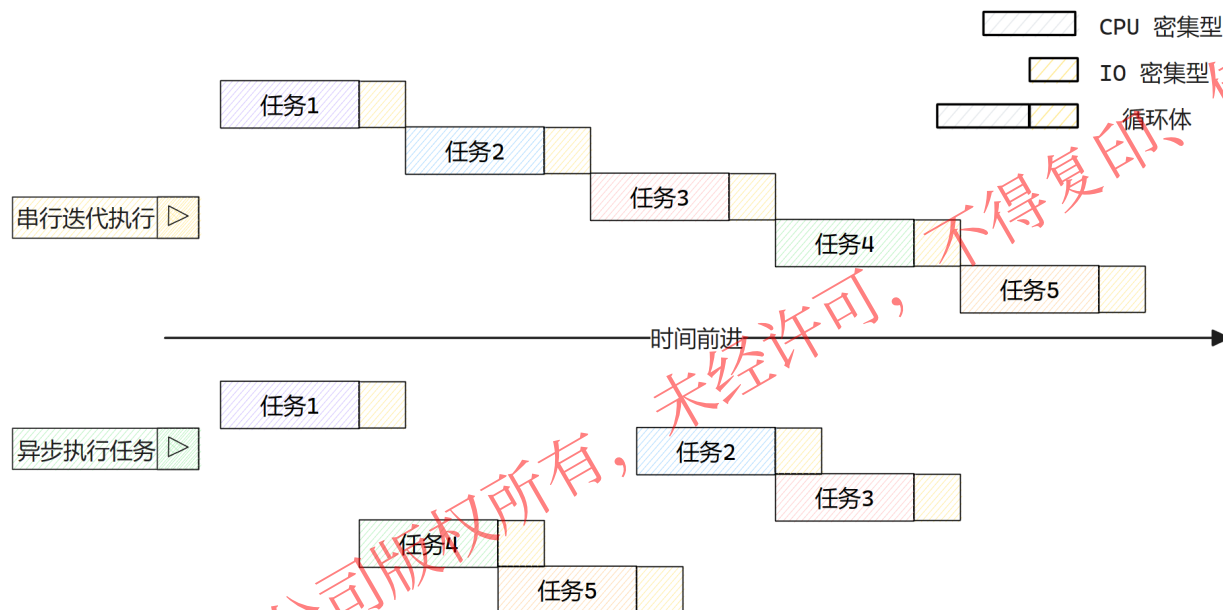
3.3 异步编程基础：并发流水线

通过异步实现流水线优化，从而在不引入更多计算资源的前提下优化代码执行时间

```
function f_sync()  
    for i in 1:5  
        busywait(2)  
        sleep(1)  
    end  
    return nothing  
end
```

同步执行

```
julia> @time f_sync()  
15.037225 seconds (567 allocations: 26.344 KiB)
```



```
function f_async()  
    @sync for i in 1:5  
        @async busywait(2)  
        @async sleep(1)  
    end  
end
```

异步执行

```
julia> @time f_async()  
11.014119 seconds (1.24 k allocations: 85.047 KiB)
```

3.3 异步编程基础：异步任务小结

通过 `@sync` + `@async` 组合构造异步任务，实现流水线优化，从而在不引入更多计算资源的前提下优化代码执行时间

常见的异步场景：

- CPU 繁忙的时候可以做一些 IO 任务
- CPU 繁忙的时候可以做一些 GPU 任务
- 同时发起多个网络请求，然后逐个处理

例：深度学习任务中存在三条主要的管线：读取数据、数据预处理、GPU 训练。
每一条管线之间使用的是独立的硬件资源。

04 任务间通信

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用

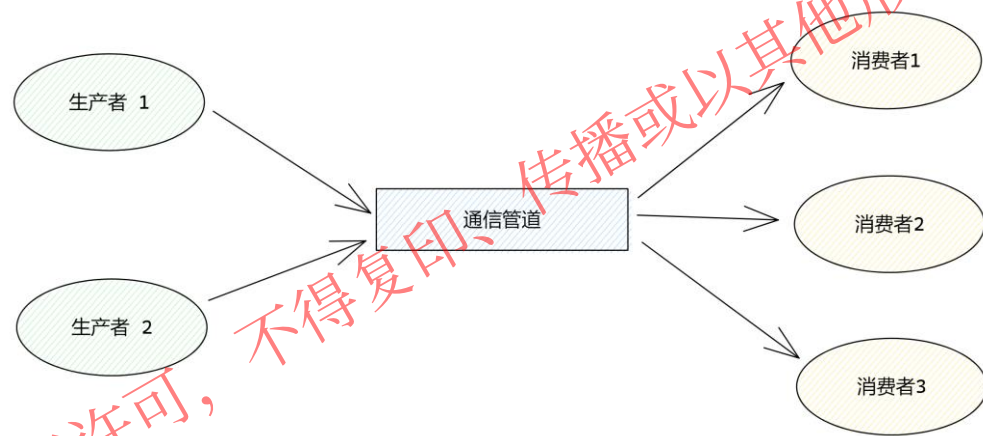
版权所有，侵权必究

4.1 任务间通信：背景

生产者-消费者模型

特征：

- 多个生产者与多个消费者之间通过消息管道进行互联
- 生产者的数目与消费者的数目并不一定相同
- 消费者并不知道东西是由哪个生产者生产的
- 生产者并不知道生产的东西将由哪个消费者使用



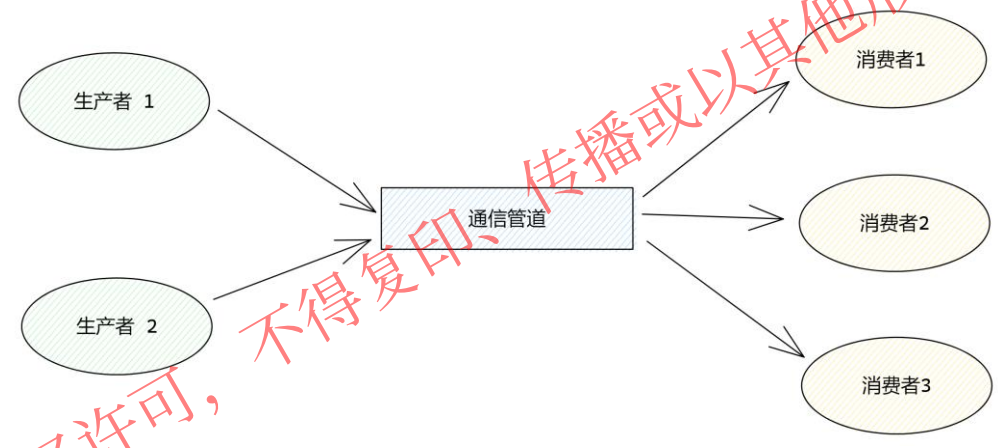
生产者-消费者模型

4.1 任务间通信：背景

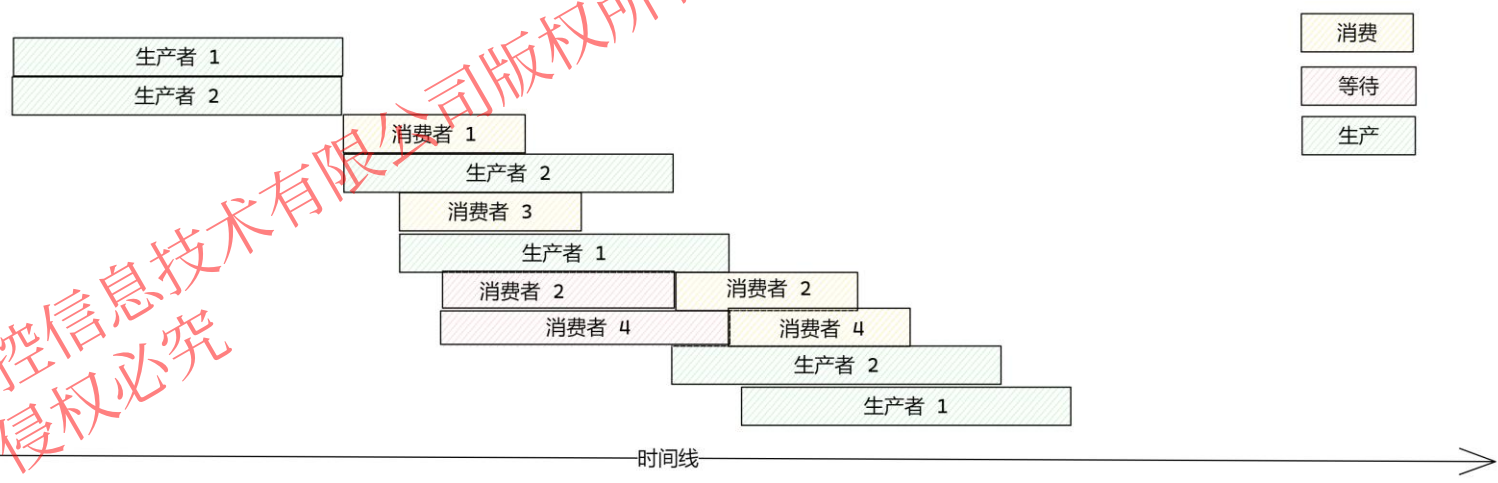
生产者-消费者模型

某一天中午：

- 厨师备餐完毕（外卖桌已满）
- 小哥1 取餐；厨师2开始下一个备餐
- 小哥3 取餐；厨师1开始下一个备餐
- 小哥2 来取餐但是发现还没有备好，等待备餐完成；
- 小哥4 来取餐但是发现还没有备好，等待备餐完成；
- 厨师2 备餐完毕；小哥2取餐；厨师2 开始下一个备餐
- 厨师1 备餐完毕；小哥1取餐；厨师1 开始下一个备餐



生产者-消费者模型

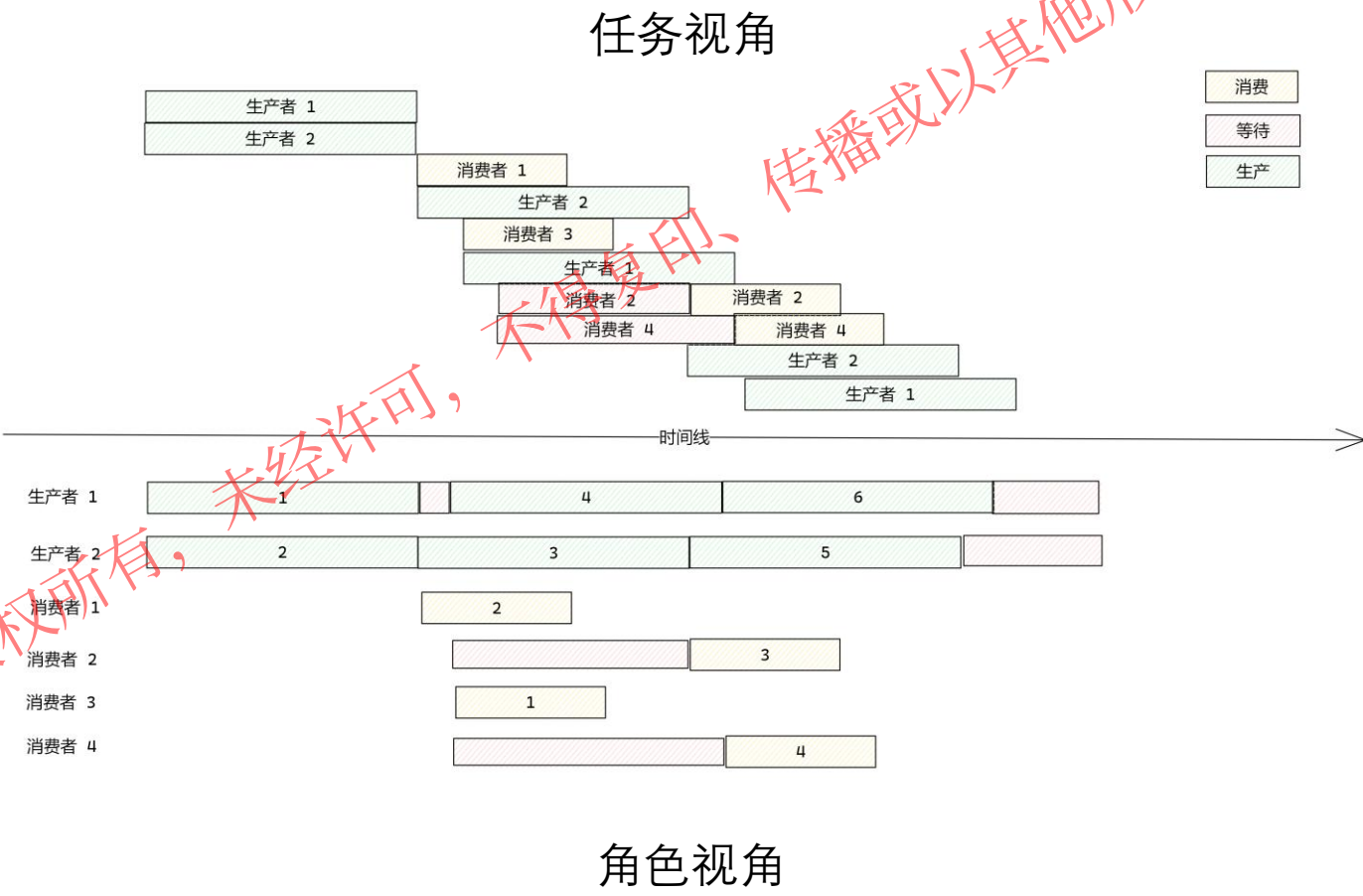


4.1 任务间通信：背景

生产者-消费者模型

某一天中午：

- 厨师备餐完毕（外卖桌已满）
- 小哥1 取餐；厨师2开始下一个备餐
- 小哥3 取餐；厨师1开始下一个备餐
- 小哥2 来取餐但是发现还没有备好，等待备餐完成；
- 小哥4 来取餐但是发现还没有备好，等待备餐完成；
- 厨师2 备餐完毕；小哥2取餐；厨师2 开始下一个备餐
- 厨师1 备餐完毕；小哥1取餐；厨师1 开始下一个备餐



4.2 任务间通信：通信管道

通过 Channel 函数可以构造一个管道对象

```
julia> ch = Channel{2}
Channel{Any}(2) (empty)

julia> put!(ch, 1) 写入数据
1

julia> put!(ch, 2)
2

julia> take!(ch) 读取数据
1

julia> take!(ch) 先进先出 FIFO (first-in, first-out) 模型
2
```

管道满的时候会阻塞写入操作

```
julia> ch = Channel{2}
Channel{Any}(2) (empty)

julia> put!(ch, 1)
1

julia> put!(ch, 2)
2

julia> put!(ch, 3) # 管道已满: 阻塞写入
```

管道空的时候会阻塞读取操作

```
julia> ch = Channel{2}
Channel{Any}(2) (empty)

julia> take!(ch) # 管道为空: 阻塞读取
```

Tip: Channel 管道是线程安全的，即当多个线程同时从写入与读取数据的时候，不会出现重复读写或漏读写问题

4.3 任务间通信：生产者-消费者模型

将管道与异步任务结合在一起

```
function produce(ch)
    id = 1
    while true
        @info "填充任务 $(id): 等待"
        put!(ch, id)
        @info "填充任务 $(id): 完成"
        id += 1

        if id > 5
            break
        end
    end
end

function consume(ch)
    wait(ch) # 等待管道中至少有一个元素之后再进入到循环
    while !isempty(ch)
        id = take!(ch)
        @info "获取任务 $id"
        sleep(1)
    end
end
```

通过 `isopen`, `isready`, `isempty` 函数用于检查管道状态
 通过 `open`, `close` 函数用于切换管道状态
 通过 `wait` 函数等待管道中存在值

单生产者-单消费者模型

```
function run_model()
    ch = Channel{2}
    @info "开启服务"
    try
        @sync begin
            @async produce(ch) # 后台任务不断往管道中填充对象
            @async consume(ch) # 前台任务不断从管道中获取对象
        end
    finally
        close(ch)
    end
    @info "关闭服务"
end
```

```
julia> run_model()
[ Info: 开启服务
[ Info: 填充任务 1: 等待
[ Info: 填充任务 1: 完成
[ Info: 获取任务 1
[ Info: 填充任务 2: 等待
[ Info: 填充任务 2: 完成
[ Info: 填充任务 3: 等待
[ Info: 填充任务 3: 完成
[ Info: 填充任务 4: 等待
[ Info: 获取任务 2
[ Info: 填充任务 4: 完成
[ Info: 填充任务 5: 等待
[ Info: 获取任务 3
[ Info: 填充任务 5: 完成
[ Info: 获取任务 4
[ Info: 获取任务 5
[ Info: 关闭服务
```

4.3 任务间通信：生产者-消费者模型

将管道与异步任务结合在一起

```
function produce(ch)
    id = 1
    while true
        @info "填充任务 $(id): 等待"
        put!(ch, id)
        @info "填充任务 $(id): 完成"
        id += 1

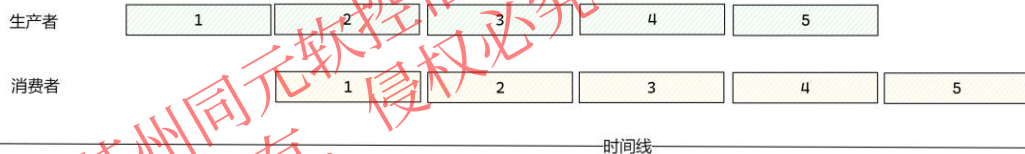
        if id > 5
            break
        end
    end
end

function consume(ch)
    wait(ch) # 等待管道中至少有一个元素之后再进入到循环
    while !isempty(ch)
        id = take!(ch)
        @info "获取任务 $id"
        sleep(1)
    end
end
```

单生产者-单消费者模型

```
function run_model()
    ch = Channel{2}
    @info "开启服务"
    try
        @sync begin
            @async produce(ch) # 后台任务不断往管道中填充对象
            @async consume(ch) # 前台任务不断从管道中获取对象
        end
    finally
        close(ch)
    end
    @info "关闭服务"
end
```

```
julia> run_model()
[ Info: 开启服务
[ Info: 填充任务 1: 等待
[ Info: 填充任务 1: 完成
[ Info: 获取任务 1
[ Info: 填充任务 2: 等待
[ Info: 填充任务 2: 完成
[ Info: 填充任务 3: 等待
[ Info: 填充任务 3: 完成
[ Info: 填充任务 4: 等待
[ Info: 获取任务 2
[ Info: 填充任务 4: 完成
[ Info: 填充任务 5: 等待
[ Info: 获取任务 3
[ Info: 填充任务 5: 完成
[ Info: 获取任务 4
[ Info: 获取任务 5
[ Info: 关闭服务
```



4.3 任务间通信：生产者-消费者模型

多生产者-多消费者模型

```
using Dates

function busywait(seconds=2)
    tstart = time_ns()
    while (time_ns() - tstart) / 1e9 < seconds
        continue
    end
    return (time_ns() - tstart) / 1e9
end

function cook(worker, name)
    @info "$(now()) - 厨师 $worker 菜品 $name: 开始准备"
    busywait(2)
    @info "$(now()) - 厨师 $worker 菜品 $name: 准备完毕"
    return name
end

function get(consumer, ch)
    @info "$(now()) - 外卖员 $consumer: 等待取餐"
    food = take!(ch)
    @info "$(now()) - 外卖员 $consumer: $food 取餐完成"
    busywait(1)
    @info "$(now()) - 外卖员 $consumer: $food 送餐完成"
end

function launch_worker(worker, food, ch)
    while isopen(ch)
        food = cook(worker, food)
        put!(ch, food)
    end
end
```

```
function run_model()
    worker1, food1 = "陈师傅", "牛肉面"
    worker2, food2 = "赵师傅", "蛋炒饭"

    consumer1 = "小李"
    consumer2 = "小陈"
    consumer3 = "小王"
    consumer4 = "小张"

    ch = Channel{2} # 外卖桌

    @info "$(now()) - 开始配餐"
    @async launch_worker(worker1, food1, ch)
    @async launch_worker(worker2, food2, ch)
    sleep(2)
    wait(ch)

    @sync begin
        @info "$(now()) - 午餐开始"
        @async get(consumer1, ch)

        sleep(0.5) # 后三个外卖员晚了一点才来
        @async get(consumer2, ch)
        @async get(consumer3, ch)
        @async get(consumer4, ch)
    end

    @info "$(now()) - 午餐结束"
end
```

4.3 任务间通信：生产者-消费者模型

多生产者-多消费者模型

```
using Dates

function busywait(seconds=2)
    tstart = time_ns()
    while (time_ns() - tstart) / 1e9 < seconds
        continue
    end
    return (time_ns() - tstart) / 1e9
end

function cook(worker, name)
    @info "$(now()) - 厨师 $worker 菜品 $name: 开始准备"
    busywait(2)
    @info "$(now()) - 厨师 $worker 菜品 $name: 准备完毕"
    return name
end

function get(consumer, ch)
    @info "$(now()) - 外卖员 $consumer: 等待取餐"
    food = take!(ch)
    @info "$(now()) - 外卖员 $consumer: $food 取餐完成"
    busywait(1)
    @info "$(now()) - 外卖员 $consumer: $food 送餐完成"
end

function launch_worker(worker, food, ch)
    while isopen(ch)
        food = cook(worker, food)
        put!(ch, food)
    end
end
```

```
function run_model()
    worker1, food1 = "陈师傅", "牛肉面"
    worker2, food2 = "赵师傅", "蛋炒饭"

    consumer1 = "小李"
    consumer2 = "小陈"
    consumer3 = "小王"
    consumer4 = "小张"

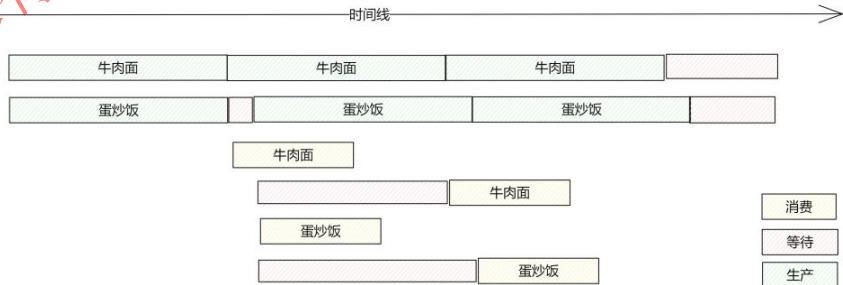
    ch = Channel{2} # 外卖桌

    @info "$(now()) - 开始配餐"
    @async launch_worker(worker1, food1, ch)
    @async launch_worker(worker2, food2, ch)
    sleep(2)
    wait(ch)

    @sync begin
        @info "$(now()) - 午餐开始"
        @async get(consumer1, ch)

        sleep(0.5) # 后三个外卖员晚了一点才来
        @async get(consumer2, ch)
        @async get(consumer3, ch)
        @async get(consumer4, ch)
    end

    @info "$(now()) - 午餐结束"
end
```



4.3 任务间通信：生产者-消费者模型

多生产者-多消费者模型

```
function run_model()
    worker1, food1 = "陈师傅", "牛肉面"
    worker2, food2 = "赵师傅", "蛋炒饭"

    consumer1 = "小李"
    consumer2 = "小陈"
    consumer3 = "小王"
    consumer4 = "小张"

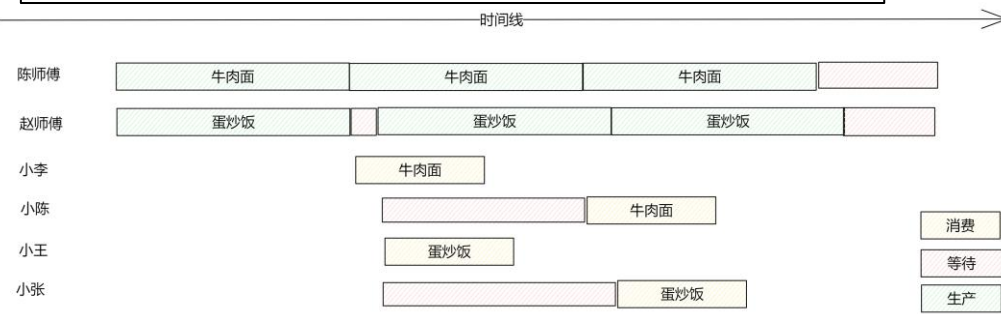
    ch = Channel{2} # 外卖桌

    @info "$(now()) - 开始配餐"
    @async launch_worker(worker1, food1, ch)
    @async launch_worker(worker2, food2, ch)
    sleep(2)
    wait(ch)

    @sync begin
        @info "$(now()) - 午餐开始"
        @async get(consumer1, ch)
        @async get(consumer2, ch)
        @async get(consumer3, ch)
        @async get(consumer4, ch)
    end

    @info "$(now()) - 午餐结束"
end
```

```
julia> run_model()
[ Info: 2023-08-19T20:51:06.787 - 开始配餐
[ Info: 2023-08-19T20:51:06.787 - 厨师 陈师傅 菜品 牛肉面: 开始准备
[ Info: 2023-08-19T20:51:06.787 - 厨师 赵师傅 菜品 蛋炒饭: 开始准备
[ Info: 2023-08-19T20:51:08.788 - 厨师 陈师傅 菜品 牛肉面: 准备完毕
[ Info: 2023-08-19T20:51:10.787 - 厨师 赵师傅 菜品 蛋炒饭: 准备完毕
[ Info: 2023-08-19T20:51:10.788 - 厨师 陈师傅 菜品 牛肉面: 开始准备
[ Info: 2023-08-19T20:51:10.788 - 午餐开始
[ Info: 2023-08-19T20:51:10.788 - 厨师 赵师傅 菜品 蛋炒饭: 开始准备
[ Info: 2023-08-19T20:51:12.788 - 厨师 陈师傅 菜品 牛肉面: 准备完毕
[ Info: 2023-08-19T20:51:14.789 - 厨师 赵师傅 菜品 蛋炒饭: 准备完毕
[ Info: 2023-08-19T20:51:14.789 - 外卖员 小李: 等待取餐
[ Info: 2023-08-19T20:51:14.789 - 外卖员 小陈: 牛肉面 取餐完成
[ Info: 2023-08-19T20:51:14.789 - 外卖员 小王: 等待取餐
[ Info: 2023-08-19T20:51:14.789 - 外卖员 小张: 等待取餐
[ Info: 2023-08-19T20:51:14.789 - 厨师 陈师傅 菜品 牛肉面: 开始准备
[ Info: 2023-08-19T20:51:15.789 - 外卖员 小李: 牛肉面 送餐完成
[ Info: 2023-08-19T20:51:15.789 - 外卖员 小陈: 蛋炒饭 取餐完成
[ Info: 2023-08-19T20:51:15.789 - 外卖员 小王: 牛肉面 取餐完成
[ Info: 2023-08-19T20:51:17.789 - 厨师 陈师傅 菜品 牛肉面: 准备完毕
[ Info: 2023-08-19T20:51:17.789 - 厨师 赵师傅 菜品 蛋炒饭: 开始准备
[ Info: 2023-08-19T20:51:18.789 - 外卖员 小陈: 蛋炒饭 送餐完成
[ Info: 2023-08-19T20:51:19.791 - 外卖员 小王: 牛肉面 送餐完成
[ Info: 2023-08-19T20:51:19.791 - 厨师 陈师傅 菜品 牛肉面: 开始准备
[ Info: 2023-08-19T20:51:19.791 - 外卖员 小张: 蛋炒饭 取餐完成
[ Info: 2023-08-19T20:51:21.790 - 厨师 赵师傅 菜品 蛋炒饭: 准备完毕
[ Info: 2023-08-19T20:51:23.790 - 厨师 陈师傅 菜品 牛肉面: 准备完毕
[ Info: 2023-08-19T20:51:24.791 - 外卖员 小张: 蛋炒饭 送餐完成
[ Info: 2023-08-19T20:51:24.791 - 厨师 赵师傅 菜品 蛋炒饭: 开始准备
[ Info: 2023-08-19T20:51:26.791 - 厨师 赵师傅 菜品 蛋炒饭: 准备完毕
[ Info: 2023-08-19T20:51:26.791 - 午餐结束
```



4.3 任务间通信：生产者-消费者模型

多生产者-多消费者模型

```
function run_model()
    worker1, food1 = "陈师傅", "牛肉面"
    worker2, food2 = "赵师傅", "蛋炒饭"

    consumer1 = "小李"
    consumer2 = "小陈"
    consumer3 = "小王"
    consumer4 = "小张"

    ch = Channel{2} # 外卖桌

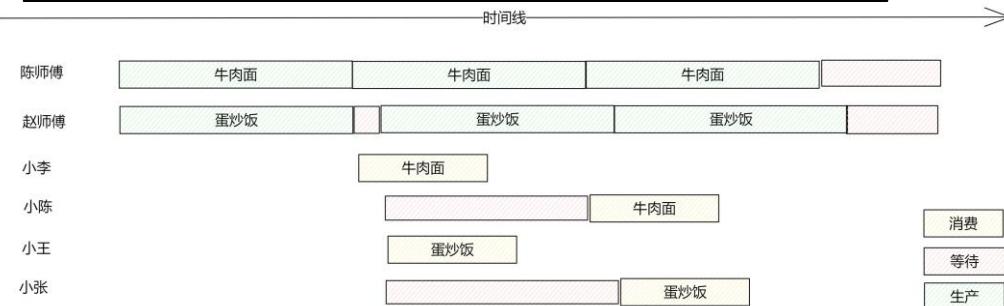
    @info "$(now()) - 开始配餐"
    @async launch_worker(worker1, food1, ch)
    @async launch_worker(worker2, food2, ch)
    sleep(2)
    wait(ch)

    @sync begin
        @info "$(now()) - 午餐开始"
        @async get(consumer1, ch)
        @async get(consumer2, ch)
        @async get(consumer3, ch)
        @async get(consumer4, ch)
    end

    @info "$(now()) - 午餐结束"
end
```

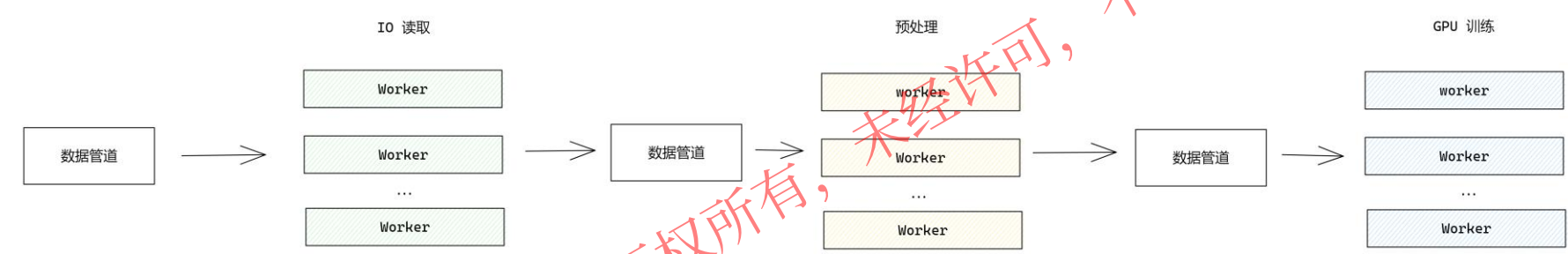
如果将所有的 @async 替换为 Threads.@spawn ,
然后以 julia -t6 的方式启动并执行 run_model() 会发生什么?

```
julia> run_model()
[ Info: 2023-08-19T20:51:06.787 - 开始配餐
[ Info: 2023-08-19T20:51:06.787 - 厨师 陈师傅 菜品 牛肉面: 开始准备
[ Info: 2023-08-19T20:51:06.787 - 厨师 赵师傅 菜品 蛋炒饭: 开始准备
[ Info: 2023-08-19T20:51:08.788 - 厨师 陈师傅 菜品 牛肉面: 准备完毕
[ Info: 2023-08-19T20:51:10.787 - 厨师 赵师傅 菜品 蛋炒饭: 准备完毕
[ Info: 2023-08-19T20:51:10.788 - 厨师 陈师傅 菜品 牛肉面: 开始准备
[ Info: 2023-08-19T20:51:10.788 - 午餐开始
[ Info: 2023-08-19T20:51:10.788 - 厨师 赵师傅 菜品 蛋炒饭: 开始准备
[ Info: 2023-08-19T20:51:12.788 - 厨师 陈师傅 菜品 牛肉面: 准备完毕
[ Info: 2023-08-19T20:51:14.789 - 厨师 赵师傅 菜品 蛋炒饭: 准备完毕
[ Info: 2023-08-19T20:51:14.789 - 外卖员 小李: 等待取餐
[ Info: 2023-08-19T20:51:14.789 - 外卖员 小李: 牛肉面 取餐完成
[ Info: 2023-08-19T20:51:14.789 - 外卖员 小陈: 等待取餐
[ Info: 2023-08-19T20:51:14.789 - 外卖员 小陈: 蛋炒饭 取餐完成
[ Info: 2023-08-19T20:51:14.789 - 外卖员 小王: 等待取餐
[ Info: 2023-08-19T20:51:14.789 - 外卖员 小王: 牛肉面 取餐完成
[ Info: 2023-08-19T20:51:14.789 - 外卖员 小张: 等待取餐
[ Info: 2023-08-19T20:51:14.789 - 外卖员 小张: 蛋炒饭 取餐完成
[ Info: 2023-08-19T20:51:14.789 - 午餐结束
```



4.3 任务间通信：生产者-消费者模型

一个 worker 可以同时既是生产者，又是消费者。



深度学习管道模型

4.4 任务间通信：小结

- 通过 Channel 管道将不同的任务联系起来，从而构造更具有实际意义的代码
- 异步及通信管道概念在多线程、多进程、GPU等各种编程场景下都非常普遍